Programming Assignment 3
POS (Parts-Of-Speech) Tagging

Issue Date : 13.04.2026
Due Date : 26.04.2026 (23.59)
Programming Language : Python

T.A. : Res. Asst. İsmail Furkan Atasoy
E-mail : ismailfurkanatasoy@cs.hacettepe.edu.tr

In this assignment, both an HMM model and a logistic regression model will be trained and tested for POS tagging using the Brown Corpus. By the end of the assignment, you will have developed two different models that can predict POS tags for a given sentence.

1 Data

The **Brown Corpus** will be used as the dataset for Part-of-Speech (POS) tagging. The dataset consists of sentences where each token is annotated with a POS tag.

Each data instance is represented as a sequence of word-tag pairs in the following format:

```
[('The', 'DET'), ('Fulton', 'NOUN'), ('County', 'NOUN'), ...]
```

where:

- **word** is a token in the sentence,
- **tag** is the corresponding Part-of-Speech label.

The dataset uses the **Universal Tagset** provided by the Natural Language Toolkit (NLTK). In this tagset, the following POS categories are used, where:

- **NOUN**: nouns and proper nouns
- **VERB**: verbs and auxiliary verbs
- **ADJ**: adjectives
- **ADV**: adverbs
- **DET**: determiners
- **ADP**: adpositions (prepositions and postpositions)
- **PRON**: pronouns
- **CONJ**: conjunctions

- **NUM**: numbers
- **PRT**: particles
- **.**: punctuation marks
- **X**: other / unknown categories

In total, the dataset contains **12 unique POS tags** under the Universal Tagset.

For modeling purposes, each sentence is treated as a sequence:

$$W = (w_1, w_2, \ldots, w_n), \quad T = (t_1, t_2, \ldots, t_n)$$

where the task is to learn a function that predicts the most likely tag sequence T given word W.

The dataset is split into training and test sets using an **80% / 20% split**. The training set is used to estimate model parameters, while the test set is used to evaluate performance.

Note: Tokens must be used exactly as they appear in the dataset. Do not apply any text preprocessing.

2 Hidden Markov Model (HMM) Tagger

In this part of the assignment, you will implement a Hidden Markov Model (HMM) for Part-of-Speech (POS) tagging. The goal is to learn probabilistic transitions between tags and probabilistic emissions from tags to words, and then decode the most likely tag sequence for a given sentence using the **Viterbi algorithm**.

You are required to complete the `HMMTagger` class provided in the template file and estimate these probabilities from the training corpus.

Task 1: Training the Model

For this part, your task is to complete the `train(self, tagged_sents)` method. During training, you must iterate over all tagged sentences and compute frequency counts for the probabilistic components of the Hidden Markov Model.

While iterating through the training data, you must update the following structures:

- `self.transitions[prev_tag][curr_tag]`: Counts how often a tag *curr_tag* follows a previous tag *prev_tag*.
- `self.emissions[tag][word]`: Counts how often a word is generated from a specific tag.
- `self.tag_counts[tag]`: Total number of times each tag appears in the corpus.

- `self.tags`: A set containing all unique POS tags observed in the training data.
- `self.vocab`: A set containing all unique words observed in the training data.

Note: We will treat the beginning of each sentence as if there is an implicit `<START>` tag preceding the first word. In other words, for the first word of each sentence, the previous tag is not empty; instead, it is set to `<START>`. It must NOT be included in self.tags, tag_counts, or any smoothing denominator involving |T| and |V|.

Task 2: Emission Probability Method

In this part, you are required to implement `get_emission_prob(self, word, tag)`. This method computes the emission probability:

$$P(w \mid t)$$

The emission probability depends on whether the word has been observed in the training vocabulary (V). To prevent zero probabilities and ensure robustness, we apply different smoothing strategies for known and unknown words.

Formally, the probability is defined as:

$$P(w \mid t) = \begin{cases} \frac{1}{C(t)+|V|} & \text{if } w \notin V \\ \frac{C(t,w)+10^{-10}}{C(t)} & \text{if } w \in V \end{cases}$$

where:

- $C(t, w)$ denotes the number of times word w appears with tag t in the training data
- $C(t)$ denotes the total number of word occurrences associated with tag t
- $|V|$ is the size of the vocabulary, i.e., the number of unique words in the training set

To ensure numerical stability, all computations are performed in log-space. Therefore, the final output is:

$$\log P(w \mid t)$$

Final Output: The function must return the log-probability value:

$$\log P(w \mid t)$$

You should use `numpy.log()` to compute this value in a numerically stable way. This is particularly important for decoding algorithms such as Viterbi, where probabilities are multiplied and can underflow without log transformation.

Task 3: Transition Probability Method

Your task is implementing `get_transition_prob(self, prev_tag, curr_tag)` for this part. This function computes:

$$P(t_i \mid t_{i-1})$$

You must:

- Count how often a tag follows another tag in the training data.

  In other words, compute transition frequencies of the form:

  $$C(t_{i-1}, t_i)$$

  which represents how many times tag t_i appears immediately after tag t_{i-1} in the training corpus.

- Apply Laplace (add-one) smoothing to handle unseen transitions and avoid zero probabilities:

  $$P(t_i \mid t_{i-1}) = \frac{C(t_{i-1}, t_i) + 1}{\sum_{t' \in T} C(t_{i-1}, t') + |T|}$$

  Here:

  - t' represents each possible tag in T,
  - $\sum_{t' \in T} C(t_{i-1}, t')$ is the total number of transitions originating from tag t_{i-1},
  - $|T|$ is the number of unique tags (used as the smoothing factor).

  This smoothing ensures that even unseen tag transitions are assigned a small non-zero probability.

- Return the logarithm of the probability:

  $$\log P(t_i \mid t_{i-1})$$

Note: Laplace smoothing must also be applied to the initial <START> state transitions exactly as it is applied to regular tag-to-tag transitions. The denominator for <START> transitions should also include $|T|$.

Task 4: Viterbi Decoding

This section assumes familiarity with the standard HMM Viterbi algorithm. The focus here is on how to implement it in practice using the structures defined in this assignment. The `viterbi(self, sentence)` method performs POS tagging by computing the most likely tag sequence using dynamic programming.

Important: To avoid numerical underflow, all probability calculations in this task must be performed in **log-space** using addition instead of multiplication.

Required Preprocessing & Initialization

Before decoding, make sure that you are using a sorted list of tags to ensure consistent indexing:

```
tags_list = sorted(self.tags)
```

You will need two 2D arrays (matrices) for dynamic programming, initialized appropriately:

- **viterbi[m, n]**: stores the best log-probability score up to state m at step n. Initialize all values to $-\infty$ (`-numpy.inf`).
- **backpointer[m, n]**: stores the index of the best previous tag. Initialize with zeros.

where m is the number of unique tags and n is the sentence length.

For the **first word** ($j = 0$), initialize the first column of the Viterbi table for each tag index i:

$$viterbi[i, 0] = \log P(t_i \mid \text{<START>}) + \log P(w_1 \mid t_i)$$

Recursion Step

For each subsequent word at position $j > 0$ and each current tag t_i, you must iterate over all previous tags t_k to compute the path scores:

$$score(k \to i) = viterbi[k, j-1] + \log P(t_i \mid t_k) + \log P(w_j \mid t_i)$$

Then, update the tables with the maximum score and its corresponding previous tag index:

- $viterbi[i, j] = \max_k score(k \to i)$
- $backpointer[i, j] = \arg\max_k score(k \to i)$

Tie-breaking Rule: If multiple tags yield the exact same maximum log-probability, you must resolve the tie by selecting the first tag based on alphabetical order. Using `numpy.argmax()` on a sequence generated from your sorted `tags_list` naturally satisfies this requirement.

Termination & Backtracking

After filling the tables, find the best final tag index:

$$\arg\max_i viterbi[i, n-1]$$

Starting from this index, reconstruct the optimal sequence by moving backwards through the `backpointer` table from right to left (from $j = n - 1$ down to 1).

Output

The `viterbi(self, sentence)` method must map the backtracked indices back to their string representations and return a list of POS tags. Formally, for an input sentence $W = (w_1, w_2, \ldots, w_n)$, the function returns the most likely sequence $T^* = (t_1, t_2, \ldots, t_n)$.

3 Logistic Regression Model

In the second part of the assignment, you will build a discriminative model for POS tagging using **Logistic Regression**. Unlike the Hidden Markov Model, which relies on the joint probabilities of sequences, Logistic Regression predicts the POS tag by learning the conditional probability of a tag given a specific set of features extracted from the word and its context.

Task 1: Feature Engineering

Your primary goal is to complete `extract_features(sentence, index)` function in the template file. This function takes a sentence (as a list of words) and the index of the target word, returning a dictionary of features. You are free to design any features you find useful.

Since POS tags are influenced by both morphology and context, you may consider implementing:

- **Word identity:** The word itself (preferably lowercase).
- **Affixes:** Prefixes/suffixes
- **Orthographic features:** Is the word capitalized? Does it contain digits?
- **Contextual features:** Features of the previous words.
- etc.

Task 2: Data Preparation and Training

To ensure the evaluation block in the `main()` function runs correctly, you must follow these implementation steps using `scikit-learn`:

1. **Prepare Training Data:** Iterate through `train_sents`. For every word in every sentence:
    - Convert the sentence into a list of words.
    - Call `extract_features()` for each word.
    - Store the resulting dictionaries in a list named `X_train`.
    - Collect the corresponding POS tags in a list named `y_train`.

2. **Define the Pipeline:** Create a `Pipeline` object named **`model`**. This pipeline should consist of two steps:
   - `'vectorizer'`: An instance of `DictVectorizer(sparse=True)`.
   - `'classifier'`: An instance of `LogisticRegression()`.
3. **Model Fitting:** You are expected to tune the model parameters if necessary to ensure convergence and improve performance. Train your pipeline by calling:

   ```
   model.fit(X_train, y_train)
   ```
4. **Prepare Test Data and Labels:** Before running the evaluation block, you must prepare both the input features and the ground truth labels for the test set by iterating through `test_sents`. Collect all feature dictionaries into a list named `X_test`. At the same time, collect the corresponding true POS tags from each sentence and store them in a separate list named `y_test`. During prediction, you should pass a list of feature dictionaries (as done in the provided evaluation code).

Note: You should continue your experiments until you achieve the highest test results.

4 Evaluation

Once you have completed the implementations for both models, you must evaluate their performance on the test set. In both provided Python templates, the evaluation blocks inside the `main()` functions are initially commented out. When your training and decoding/prediction functions are ready, you need to uncomment these blocks to run the evaluation pipeline.

Important: You must not add any additional `print` statements beyond those already provided in the template. All existing and commented `print` statements in the template should remain active (uncommented) when submitting your final solution.

HMM Evaluation Expectations

Because the mathematical formulations, smoothing constants (e.g., $\epsilon = 1e - 10$), and tie-breaking rules for the Viterbi algorithm are strictly defined in this assignment, your HMM implementation is completely deterministic.

This means there is only one uniquely correct output. If your implementation follows all the instructions precisely, you should obtain **exactly** the same accuracy and sequence predictions as the given output file.

Logistic Regression Evaluation Expectations

Unlike the HMM assignment, the Logistic Regression model is highly open-ended. Your final results will depend entirely on two factors:

- The features you engineered and implemented in the `extract_features()` function.
- The hyperparameters you chose for the `LogisticRegression` classifier.

Therefore, **it is completely normal and expected** that you will obtain different accuracy scores and classification reports. For this section, you are not expected to match a specific accuracy number. Instead, your work will be evaluated based on the logical soundness and creativity of your feature set, the correct usage of the `scikit-learn` pipeline, and whether your model achieves a reasonable baseline performance for a POS tagging task.

Submission

- You will submit your programming assignment using the **HADI** system. You have to upload a single **zip file** (`.zip`, `.gzip`, or `.rar`) containing **2 Python files** (`.py` files): one for the Hidden Markov Model (HMM) solution and one for the Logistic Regression (LR) solution.

- The name of your Python files should be:

- `pa3_FullName_hmm.py` for the Hidden Markov Model solution
- `pa3_FullName_lr.py` for the Logistic Regression solution

Replace `FullName` with your actual name.

- The name of your zip file should match: `pa3_FullName` with a corresponding extension (`.zip`, `.gzip`, or `.rar`).

Late Policy:

- You must submit your programming assignment **before its due date**.
- You may submit your assignment up to **three days late**, but with a penalty:
  - **1 day late:** 10% penalty
  - **2 days late:** 20% penalty
  - **3 days late:** 30% penalty

DO YOUR PROGRAMMING ASSIGNMENTS YOURSELF!

- **Do not share your programming assignments with your friends.**
- **Do not complete your entire assignment using AI tools and comment on the parts where you have received support from AI tools.**
- **Cheating will be punished.**